Programming Assignment 1
Dhurandhar Programming

CS683: Advanced Computer Architecture, Autumn 2026
Computer Science and Engineering
Indian Institute of Technology, Bombay
CASPER group: https://casper-iitb.github.io/

* *Disclaimer: The memes included in this document are intended solely for fun learning. They are not meant to offend, mislead, or be taken as factual information. Please enjoy them in the spirit of fun learning.*

Most programs that a programmer writes are driven primarily by functionality. When it comes to optimizing performance, they often experiment with various algorithms to achieve the best possible execution time. However, the underlying hardware that runs the code is frequently overlooked, leaving a significant portion of potential optimizations untapped. To truly optimize performance, a programmer must understand how the software interacts with the hardware. Only by gaining insight into both the code and the architecture can they meaningfully improve their programs' performance.

This assignment aims to give you a taste of what happens when you optimize code with the underlying architecture in mind. In particular, we will explore how to make better use of the cache by improving **data reuse**.

NOTE:

Just a friendly reminder: If you think copying code is a clever shortcut, think again. It's not only easily spotted but also a great way to miss out on the chance to actually learn something. Why not impress us with your own work?

- You need to have Intel-based x86 machines to implement software prefetching.
- Make changes to the base code only. Do not use a different implementation of the base code.
- No compiler or any other optimizations should be used. Points will be deducted for using any additional optimization techniques other than the ones mentioned in the assignment.

The assignment is divided into two tasks, each having its subparts. The task structure and its respective points are shown below:

Main Tasks		
Task 1	2D Convolution	6 points
Task 2	Matrix multiplication for inference	9 points
Total		15 points

Task 1

2D Convolution — The Dhurandhar Way

This assignment focuses on optimizing the performance of 2D convolution using various optimization techniques. You will implement and analyze multiple methods, measure their impact on performance, and gain a deeper understanding of the underlying principles that drive performance improvements.

Note: Please refrain from modifying any part of the code other than the 2D convolutions function.

List of Tasks (6 points)		
1A	Loop unrolling and reordering	1 point
1B	Tiling	1 point
1C	SIMD	2 points
1D	Sabka saath sabka vikaas	2 points

Task 1A: Loop unrolling and reordering



Analyze the provided 2D convolution code and implement loop reordering and unrolling optimizations to improve execution time.

By the end of this task, you should be able to answer the following questions:

1. What motivated you to make changes to your code?
2. What considerations did you take into account when implementing those changes?
3. How effective are your changes? (Which metric will you use to quantify effectiveness and why?)

Task 1B: Tiling



Deliverables

1. ***Profiling baseline 2D convolution code:***

   - Find out the size of the **L1 Data (L1-D) cache** on your system.
   - Profile the baseline (naive) 2D convolution implementation and report the **L1-D cache misses per kilo instructions (MPKI)**.

     Hint: use a tool called perf

2. ***Implementing Tiled 2D convolution***

   - Implement a **tiled** version of 2D convolution.
   - Experiment with different **tile sizes**.
   - For each tile size, measure the **L1-D cache MPKI** to evaluate cache behavior.
   - Identify the **tile size** that gives the best performance on your hardware in terms of cache efficiency and execution time.

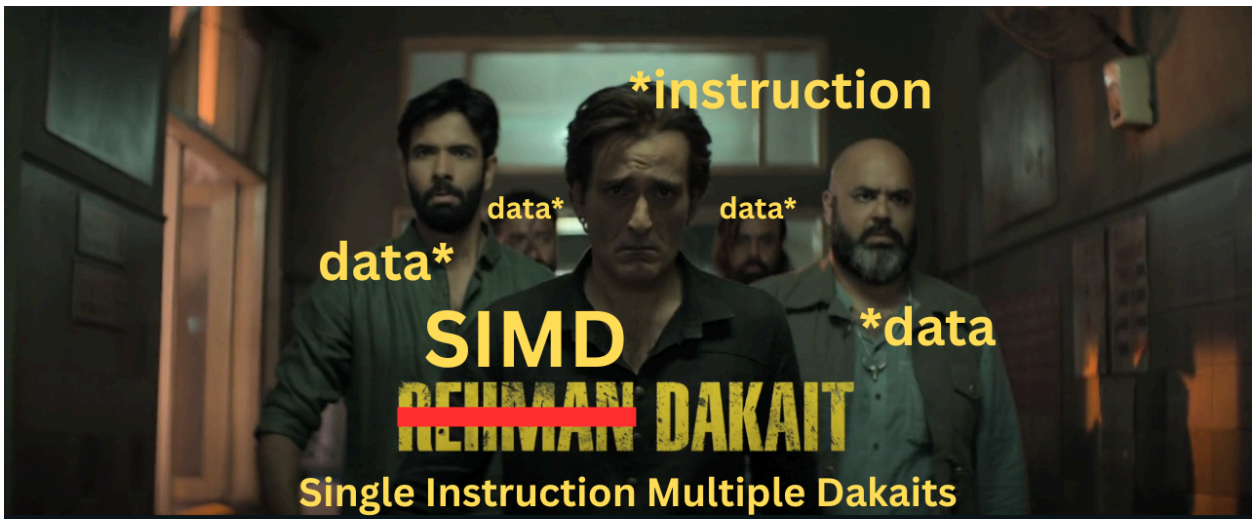
3. ***Collecting and analyzing different metrics of interest:***

- Test your tiled implementation on matrices of various sizes.
- Calculate the **speedup** achieved by the tiled version compared to the baseline implementation for each matrix size.
- Create plots to visualize the performance trends:
    - **L1-D MPKI vs. Matrix Size** for different tile sizes.
    - **Speedup vs. Matrix Size** for different tile sizes.
    -

Answer the following questions:

1. Report the changes in **L1-D MPKI** observed when moving from naive to tiled 2D convolution. Justify your observations.
2. How did **L1-D MPKI** vary across different matrix sizes and tile sizes? Explain your findings in terms of the cache hierarchy and working set sizes.
3. Did you achieve a **speedup**? If yes, quantify the improvement and identify the contributing factors. If not, analyze the limiting factors and propose possible solutions.

Task 1C: SIMD



In this task, you will utilize SIMD (Single Instruction, Multiple Data) instructions to vectorize 2D convolution and analyze their impact on instruction count and overall performance.

Deliverables

1. ***Baseline Profiling:***
    - Report the **number of instructions** executed for the **naive** 2D convolution implementation.
2. ***SIMD Implementation:***
    - Modify your 2D convolution to leverage **SIMD instructions**.
    - Implement versions using **128-bit**, **256-bit**, and (if available) **512-bit** SIMD registers.
3. ***Instruction Count and Performance Analysis:***
    - Report the **number of instructions** executed when running only the **SIMD version**.

- **Compare performance** between the naive and SIMD implementations by calculating the **speedup** achieved.

4. ***Multi-Size Evaluation:***

   - Run your implementations on matrices of various sizes.
   - Analyze how performance and speedup vary with **matrix size** and **SIMD register width**.

5. ***Visualization:***

   - Create plots showing the **speedup** for different matrix sizes and SIMD widths.
   - Choose matrix sizes that allow you to **clearly observe trends and draw meaningful conclusions**.

Deliverables

Answer the following:

1. Report the change in the number of instructions you observed when moving from the naive to the SIMD implementation. Justify your observations.
2. Did you achieve any speedup? If so, how much, and what contributed to it? If not, what were the reasons?
3. Which SIMD intrinsics did you use? Justify your choice of functions.

Task 1D: Sabka saath sabka vikaas

In this part of the assignment, you are expected to demonstrate how combining multiple optimization techniques, such as tiling, SIMD vectorization, loop unrolling, and cache-aware programming, can lead to better performance improvements than applying each technique in isolation.

Final performance summary for 2D convolution:

Implement a version of 2D convolution that combines two or more of the previously explored optimization techniques.

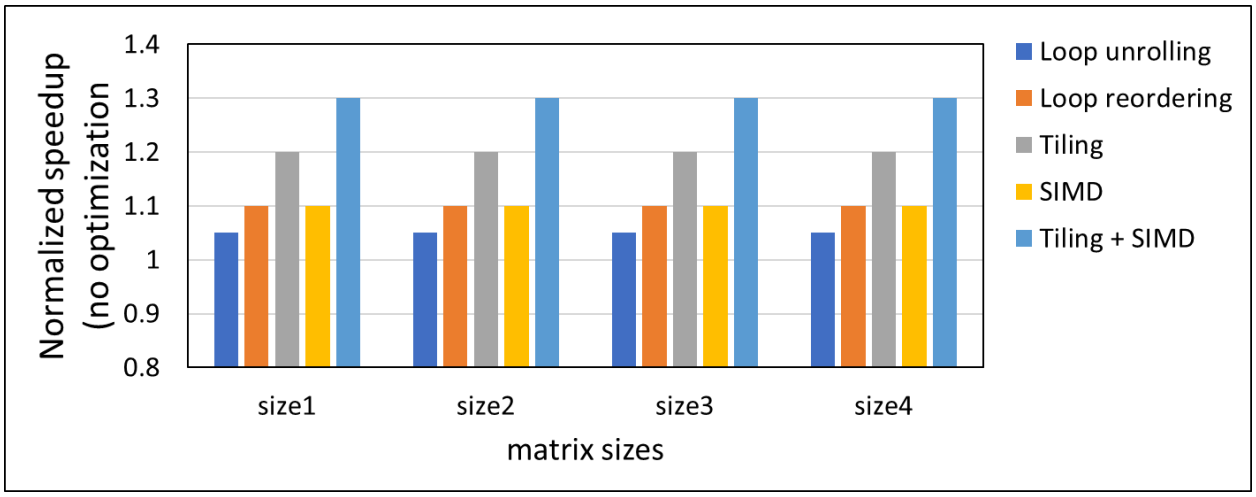
Demonstrate how these techniques complement each other to provide synergistic performance gains, i.e., the combined benefit is greater than the sum of individual optimizations.

Include a plot that compares the best performance achieved by each optimization technique:

- Loop unrolling
- Loop reordering
- Tiling
- SIMD
- Tiling + SIMD

Plot this comparison against varying matrix sizes. Refer to **Figure 1.1** for reference.

Figure 1.1:



Note: This is an example plot, so we have only shown five optimization techniques. You can try different combinations and expand the plot accordingly.

Task 2

Matrix multiplication for inference

Matrix multiplication is an algebraic operation that combines two matrices to produce a third matrix. Matrix multiplication is the backbone of modern LLM inference.

In this section, you will optimize the matrix multiplication operation using optimization techniques such as software prefetching and SIMD.

List of Tasks (9 points)		
2A	Software prefetching	5 points
2B	SIMD	1 point
2C	Software prefetching + SIMD	3 points

Task 2A: Software prefetching

In this part of the assignment, you will be optimizing the SGEMM (Single-Precision General Matrix Multiplication) code using the software prefetching technique. You will tune various software prefetching parameters for the SGEMM operation by analyzing trends across different CPU performance metrics. Additionally, you will study the impact of hardware prefetchers on the performance of the SGEMM operation.



You will be tuning two key parameters of software prefetching:

1. Prefetch Distance
2. Cache Fill Level

Both **_builtin_prefetch** and **_mm_prefetch** allow you to control these parameters during software prefetching.

Additionally, you will use the **sw_prefetch_access** event in *perf* to track the number of software prefetch requests issued during the execution.

Deliverables

1. *Analyze the Impact of the matrix size*:

- Increase the size of the matrices and observe how software prefetching performance changes.

2. *Analyze CPU Metrics*:

For different matrix sizes and different software prefetching parameters, analyze the trends in key CPU metrics (use the *perf* tool):

- L1D Cache Misses
- L2 Cache Misses
- LLC (Last Level Cache) Misses

3. *Collect Execution Time and Compute Speedup*:

- Measure the execution time of the SGEMM operation with and without software prefetching.
- Compute the speedup as follows:

$$Speedup = \frac{Execution\ Time\ without\ Software\ Prefetching}{Execution\ Time\ with\ Software\ Prefetching}$$

- Analyze how the speedup varies with matrix size and software prefetching parameters.

4. *Identify Optimal Parameters*:

Determine the optimal prefetch distance and optimal cache fill level based on your observations.

5. *Study the Effect of Hardware Prefetchers*:

Enable and disable hardware prefetchers, and analyze their impact on the performance of the SGEMM operation.

Deliverables

Identify the values of different metrics for different matrix sizes, prefetching distances, and cache fill levels. Analyze them carefully and state your insights for the question below.

1. What trend do you observe in speedup with different matrix sizes?
2. What is the best prefetch distance?
3. At what cache fill level do you achieve the maximum speedup

Task 2B: SIMD (Single Instructor Multiple Deadlines)

Utilize SIMD (Single *Instruction*, Multiple Data) instructions to parallelize the SGEMM operation, and analyze their impact on both instruction count and overall performance.

Deliverables

1. *Analyze the Impact of SIMD Instruction Width*:

Experiment with different SIMD instruction widths provided by Intel intrinsics (e.g., 64-bit, 128-bit, 256-bit), depending on your machine's capabilities, and observe how each affects performance.

2. *Analyze the Impact of matrix size*:

Try different matrix sizes in combination with different SIMD widths.

3. *Analyze Instruction Count*:

Use the perf tool (or any suitable performance analysis tool) to measure the instruction count for different SIMD widths and matrix sizes.

4. *Collect Execution Time and Compute Speedup*:

- Compute the speedup as follows:

$$Speedup = \frac{Execution\ Time\ without\ SIMD}{Execution\ Time\ with\ SIMD}$$

- Analyze how the speedup varies with different matrix sizes and different software prefetching parameters.

Consider **Table 2.1** for reference. Identify the values of different metrics for different matrix sizes and SIMD widths. Analyze them carefully and state your insights for the question below.

1. What trends do you observe in speedup for different combinations of matrix sizes and SIMD widths?
2. For which SIMD width do you achieve the maximum speedup?

Note: This is just an example table, so we have provided entries for only a few matrix sizes and SIMD widths. You should expand and modify it as needed based on your analysis.

Table 2.1:

Metrics		**Matrix sizes**			**SIMD width (bits)**		
		(M1,N1,K1)	(M2,N2,K2)	. . .	128	256	512
No SIMD	Instructions						
	Execution time						
SIMD	Instructions						
	Execution time						
Speedup (normalized to no SIMD)							

Task 2C: Software prefetching + SIMD

In this task, you will combine both software prefetching and SIMD techniques to optimize the SGEMM operation. You are required to repeat all the analyses you performed individually in Task 2A (Software Prefetching) and Task 2B (SIMD), but now with both optimizations applied together.

Deliverables

1. *All the objectives from Task 2A and Task 2B*.

Final performance summary for matrix multiplication operation:

Include a plot that compares the best performance achieved by each optimization technique:

- Software Prefetching
- SIMD
- Software Prefetching + SIMD

Plot this comparison across varying matrix sizes.

APPENDIX

Content	
1	*perf* tool
2	Software prefetching functions
3	SIMD functions

1. *perf* tool

- The following are the links where you can find details about **perf**: https://perfwiki.github.io/main/
- Demo slides: https://docs.google.com/presentation/d/1w_acNFVTud5YkASOEUEnuKXHX_4-nhftENyV0Jji3og/edit?usp=sharing

2. Software prefetching functions

2.1. *mm_prefetch*

- To implement software prefetching, you will use '**_mm_prefetch**.'
- **_mm_prefetch** is an intrinsic function provided by Intel that prefetches data into the cache to enhance memory access efficiency. It enables programmers to give the processor advanced notice about which memory locations will be accessed soon, reducing cache misses and improving performance.

- The function is part of Intel's SSE (Streaming SIMD Extensions) and is highly optimized for Intel architectures. It is especially effective when used with SIMD (Single Instruction, Multiple Data) operations.
- The following are the links where you can find details about **_mm_prefetch**:
    - https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html#ig_expand=5152&text=prefetch
    - https://stackoverflow.com/questions/46521694/what-are-mm-prefetch-locality-hints

2.2. *builtin_prefetch*

3. SIMD functions

Here are a few basic points to get started with using SIMD (Single Instruction, Multiple Data) instructions.

3.1. *Understanding SIMD Registers:*

- SIMD (Single Instruction, Multiple Data) allows the same operation to be performed on multiple data points simultaneously, which can significantly speed up computations.
- _m128d and _m256d represent 128-bit and 256-bit SIMD registers, respectively. These registers can hold multiple double-precision (64-bit) floating-point numbers.
- _m128d can hold two double-precision floating-point numbers.
- _m256d can hold four double-precision floating-point numbers.
- There is also _m512d, which can hold eight double-precision floating-point numbers. You can check if your system supports this by doing:

```
lscpu | grep avx512
```

If this yields some flags like avx512*, there you go!

3.2. *Loading Data into SIMD Registers:*

- Before performing any SIMD operations, data must be loaded into these registers.
- One of the ways to load data is using functions like _mm256_loadu_pd (for _m256d registers), which loads four double-precision floating-point values from memory into a SIMD register. The "u" in loadu stands for "unaligned," meaning the data does not need to be aligned to a specific boundary in memory.

3.3. *Performing SIMD Operations:*

- Once the data is loaded into SIMD registers, you can perform arithmetic operations on these registers.
- Functions like _mm256_add_pd and _mm256_mul_pd allow you to perform addition and multiplication, respectively, on the elements in the registers.

You can refer to all these functions here: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html.

Submission Instructions

- You should submit a single tar.gz file with the name **rollno1_rollno2_rollno3_rollno4_pa1.tar.gz** on Moodle. (**NOTE**: rollno1 **<** rollno2 < rollno3 < rollno4)
- Only one member should submit it on Moodle.
- The folder structure within the tar.gz file should be as shown below. Place the files in the appropriate folders for each task.
- Link to boilerplate code. Maintain a professional GitHub repository with a proper commit history. Commits pushed after the submission deadlines will not be considered for evaluation.

```
pa1-dhurandhar-microarchitecture/
├── task1/                                  # Task 1: 2D convolution optimization
│   ├── src/
│   │   ├── conv_reorder.cpp                # Loop reordering
│   │   ├── conv_unroll.cpp                 # Loop unrolling
│   │   ├── conv_tile.cpp                   # Cache tiling
│   │   ├── conv_simd.cpp                   # SIMD (AVX2)
│   │   └── conv_optimized.cpp              # All combined (graded on speed)
│   └── task1_report.pdf                    # Speedup plots vs. matrix size and kernel size,
│                                            # with justification for each technique's gain/loss
├── task2/                                  # Task 2: Matrix multiplication vs llama.cpp
│   ├── src/
│   │   ├── matmul_simd.cpp                 # SIMD (AVX2)
│   │   ├── matmul_prefetch.cpp             # Software prefetch
│   │   └── matmul_optimized.cpp            # All combined (graded, injected into llama.cpp)
│   └── task2_report.pdf                    # Speedup plots vs. matrix size, prefetch degree,
│                                            # prefetch level (T0/T1/T2/NTA), and SIMD width
│                                            # (128/256/512-bit), with justification
├── plots/                                  # Supporting figures referenced by the reports
│   ├── task1_speedup_vs_size.png
│   ├── task1_speedup_vs_kernel.png
│   ├── task2_speedup_vs_size.png
│   ├── task2_prefetch_degree.png
│   ├── task2_prefetch_level.png
└── README.md                               # Assignment overview
```

Deadline

Deadline: 07/09/2026



Vivas: Post midsem

